Distributed Computing

Introduction

Naveen Kumar

Assistant Professor
SVNIT, Surat

January 13, 2026

Table of Contents

1. Introduction
2. Design Goals of Distributed Systems
   - Easy access to resources
   - Distribution Transparency
   - Being Open
     - Separating Policy from Mechanism
   - Being Scalable
3. Types of Distributed Systems
   - Distributed Computing System
     - Cluster Computing
     - Grid Computing
     - Cloud Computing
   - Distributed Information Systems
     - Enterprise Application Integration
   - Pervasive Systems
4. Summary

Table of Contents

1. Introduction
2. Design Goals of Distributed Systems
   - Easy access to resources
   - Distribution Transparency
   - Being Open
     - Separating Policy from Mechanism
   - Being Scalable
3. Types of Distributed Systems
   - Distributed Computing System
     - Cluster Computing
     - Grid Computing
     - Cloud Computing
   - Distributed Information Systems
     - Enterprise Application Integration
   - Pervasive Systems
4. Summary

Introduction

- **Early computers (1945–1985):**
  - Large, expensive, room-sized mainframes
  - Operated as standalone systems
  - Used mainly by governments, universities, and large industries
- **Major transformation since the mid-1980s** due to:
  - **Microprocessors:**
    - Evolution from 8-bit to 16-, 32-, and 64-bit processors
    - Emergence of multicore CPUs
  - **High-speed computer networks:**
    - Local Area Networks (LANs)
    - The global Internet
- Modern machines rival old mainframes at a fraction of the cost
- Smartphones and plug computers are fully networked computing systems

Distributed Systems Overview

- A **distributed system** consists of **multiple computers** connected by a **network**
- **System size** can vary:
  - **Small-scale systems** with a **few nodes**
  - **Large-scale systems** with **millions of nodes**
- **Nodes** may be **geographically dispersed** across **cities or continents**
- **Communication networks** may be:
  - **Wired** (Ethernet, fiber)
  - **Wireless** (Wi-Fi, cellular)
  - **Hybrid combinations**
- **Highly dynamic behavior**:
  - Nodes can **join or leave** at any time
  - **Failures are normal** rather than exceptional

What Is a Distributed System?

Definition

A distributed system is a collection of autonomous computing elements that appears to its users as a single coherent system.

- **Autonomous nodes:**
  - Each node has its own processor, memory, and operating system
  - No shared physical memory between nodes
- Nodes communicate exclusively through message passing
- Cooperation among nodes creates the illusion of a single system

Characteristic 1: Autonomous Nodes

- Nodes operate independently and make local decisions
- Communication occurs via explicit message exchange
- No global clock exists:
  - Time synchronization is difficult
  - Events cannot be totally ordered easily
- Distributed systems must manage:
  - Group membership (who is part of the system)
  - Admission control (open vs closed systems)
  - Authentication and trust management

Overlay Network Organization

- Many distributed systems use **overlay networks**
- Logical communication structure built on top of physical networks
- Overlay connections may differ from physical network topology
- Two major overlay types:
  - **Structured overlays:**
    - Organized (trees, rings, distributed hash tables)
  - **Unstructured overlays:**
    - Random or ad-hoc neighbor connections
- Peer-to-Peer (P2P) systems are common overlay-based systems

Characteristic 2: Single Coherent System

- The system should behave consistently regardless of:
  - Physical location
  - Time of access
  - Access point used
- Users should be unaware of:
  - Where processes execute
  - Where data is physically stored
- This goal is known as **distribution transparency**
- Partial failures are unavoidable and must be handled gracefully

Middleware in Distributed Systems

- Middleware is a software layer placed above operating systems
- Provides a uniform interface to distributed applications
- Masks heterogeneity of hardware, operating systems, and networks
- Acts as a network-wide operating system

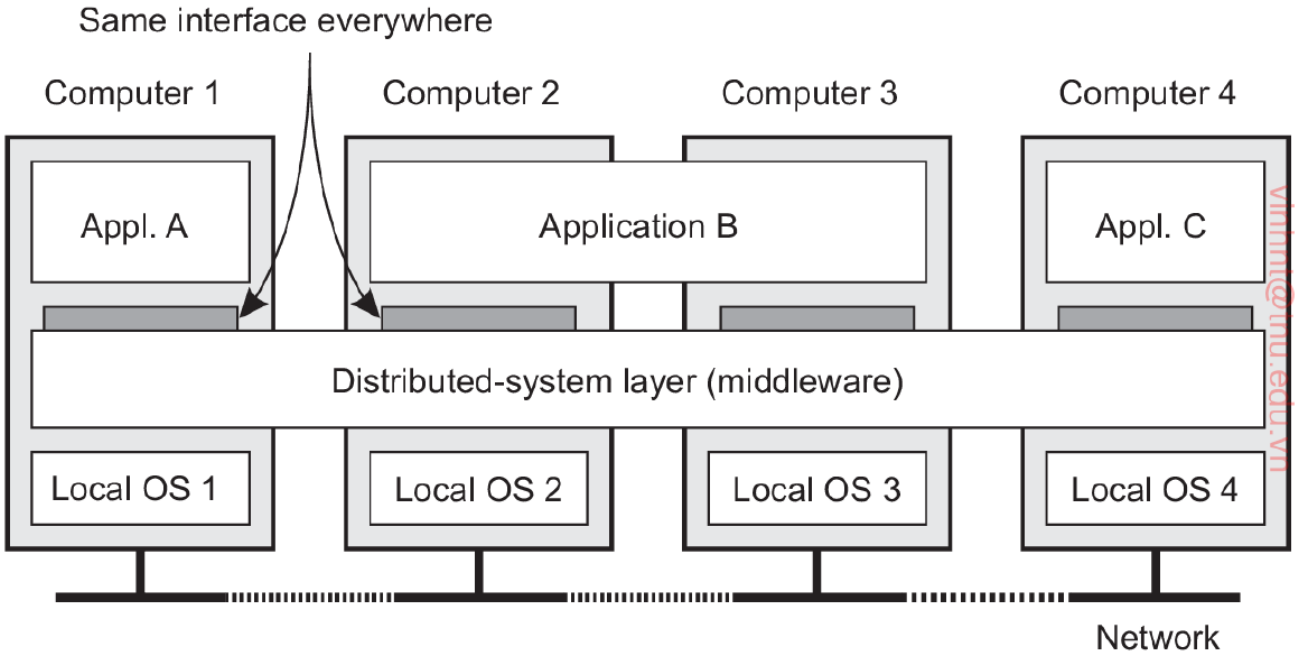


Figure 1: A distributed system organized through middleware, providing a uniform interface across multiple machines.

Middleware Services

- **Communication:**
  - Remote Procedure Call (RPC)
  - Message-oriented communication
- **Transactions:**
  - Atomic and consistent execution of operations
- **Service composition:**
  - Web services and mashups
- **Security and accounting:**
  - Authentication, authorization, billing
- **Reliability and fault tolerance:**
  - Failure detection and recovery

Table of Contents

1. Introduction
2. Design Goals of Distributed Systems
   - Easy access to resources
   - Distribution Transparency
   - Being Open
     - Separating Policy from Mechanism
   - Being Scalable
3. Types of Distributed Systems
   - Distributed Computing System
     - Cluster Computing
     - Grid Computing
     - Cloud Computing
   - Distributed Information Systems
     - Enterprise Application Integration
   - Pervasive Systems
4. Summary

Design Goals

- Designing distributed systems is **complex, expensive, and risky**
- Merely being able to build a distributed system does **not** justify building one
- Distributed systems are worthwhile only if they provide clear benefits
- A well-designed distributed system should satisfy the following goals:
  - **Easy access to resources**
  - **Distribution transparency**
  - **Openness**
  - **Scalability**
- These goals guide architectural and design decisions

Table of Contents

1. Introduction
2. Design Goals of Distributed Systems
   - Easy access to resources
   - Distribution Transparency
   - Being Open
     - Separating Policy from Mechanism
   - Being Scalable
3. Types of Distributed Systems
   - Distributed Computing System
     - Cluster Computing
     - Grid Computing
     - Cloud Computing
   - Distributed Information Systems
     - Enterprise Application Integration
   - Pervasive Systems
4. Summary

Easy access to resources

- **Primary objective:** make remote resources easy to access and share
- Resources in a distributed system may include:
  - Hardware peripherals (printers, sensors)
  - Storage systems (disks, databases)
  - Data and files
  - Software services and communication networks
- Users and applications should access these resources as if they were local
- Resource sharing forms the foundation of most distributed systems

Motivation for Resource Sharing

- **Economic motivation:**
  - Sharing expensive, high-end resources is cheaper than replicating them
  - Centralized maintenance improves reliability and manageability
- **Collaboration motivation:**
  - Internet enables file transfer, email, and document sharing
  - Supports audio and video communication
- Enables geographically distributed teams to work together
- Widely adopted in multinational software development projects

Peer-to-Peer Resource Sharing

- Peer-to-peer (P2P) systems demonstrate large-scale resource sharing
- In P2P systems:
  - Each node acts as both a client and a server
  - No single centralized authority exists
- Commonly used for:
  - File and media distribution
  - Software updates
  - Backup and data synchronization
- P2P systems scale well and avoid single points of failure

Shared Folders Worldwide

- Modern Web-based services provide globally shared folders
- Files placed in a shared folder are:
  - Automatically synchronized across users and devices
  - Updated transparently in the background
- To users, shared folders appear identical to local folders
- Physical storage location of data is completely hidden
- Services support multiple platforms and operating systems

Table of Contents

1. Introduction
2. Design Goals of Distributed Systems
   - Easy access to resources
   - Distribution Transparency
   - Being Open
     - Separating Policy from Mechanism
   - Being Scalable
3. Types of Distributed Systems
   - Distributed Computing System
     - Cluster Computing
     - Grid Computing
     - Cloud Computing
   - Distributed Information Systems
     - Enterprise Application Integration
   - Pervasive Systems
4. Summary

Making Distribution Transparent

- Hide physical distribution of processes and resources
- Distribution should be invisible to:
  - Users
  - Applications
- Transparency simplifies system usage and programming

Types of Distribution Transparency

- Different aspects of transparency apply to objects
- Objects can be:
  - Processes
  - Resources

Transparency	Description
Access	Hide differences in data representation and how an object is accessed
Location	Hide where an object is located
Relocation	Hide that an object may be moved to another location while in use
Migration	Hide that an object may move to another location
Replication	Hide that an object is replicated
Concurrency	Hide that an object may be shared by several independent users
Failure	Hide the failure and recovery of an object

Figure 2: Different forms of transparency in a distributed system (see ISO [1995]). An object can be a resource or a process.

Access Transparency

- **Goal:** Access remote resources *in the same way* as local ones
- Hides differences in:
  - Data representation (e.g., byte order, data formats)
  - Access methods (local calls vs remote calls)
  - Underlying machine architectures
- Masks operating system differences:
  - File naming conventions
  - File and resource operations
  - Low-level communication mechanisms
- **Result:** Applications use a *uniform interface* regardless of location or platform

Location and Relocation Transparency

- **Location transparency:**
  - Users and applications cannot tell where an object is physically located
  - Access is based on *logical names*, not physical addresses
- Example:
  - URLs hide the physical location of servers
- **Relocation transparency:**
  - Objects may be moved to a different location while in use
  - Movement is completely invisible to users

Migration Transparency

- **Goal:** Allow objects or processes to move *by user request*
- Migration does not disrupt:
  - Ongoing communication
  - Active operations
- Typical examples:
  - Mobile phone calls while users move
  - Tracking goods during transportation
  - Mobile devices in teleconferencing

Replication Transparency

- **Goal:** Improve availability and performance using replicas
- Resources are replicated to:
  - Survive failures
  - Reduce access latency
- Users should not be aware of:
  - Multiple copies of the same resource
- All replicas share the same logical name
- Requires location transparency

Concurrency Transparency

- **Goal:** Allow safe sharing of resources
- Multiple users may access the same resource simultaneously
- Users should not notice:
  - Conflicts
  - Interference from other users
- Requires consistency guarantees
- Achieved using:
  - Locking mechanisms
  - Transactions

Failure Transparency

- **Goal:** Hide failures from users and applications
- System should:
  - Detect failures
  - Recover automatically
- Extremely difficult to achieve in practice
- Main challenge:
  - Distinguishing a failed process from a slow one

Limits of Distribution Transparency

- Full transparency is often impossible
- Physical limitations:
  - Speed of light
  - Network latency
- High transparency can:
  - Reduce system performance
  - Increase response time

Why Full Transparency Can Be Problematic

- Completely hiding distribution may confuse developers
  - Network delays and failures are hidden
  - Programs may behave unpredictably
- **Remote Procedure Calls (RPC)**
  - Look similar to local function calls
  - Behave very differently under failures
- **Message-based communication**
  - Makes failures and delays explicit
  - Easier to reason about in distributed systems
- **Copy-before-use principle**
  - Data is copied locally before use
  - Updates create new versions of data

Table of Contents

1. Introduction
2. Design Goals of Distributed Systems
   - Easy access to resources
   - Distribution Transparency
   - Being Open
     - Separating Policy from Mechanism
   - Being Scalable
3. Types of Distributed Systems
   - Distributed Computing System
     - Cluster Computing
     - Grid Computing
     - Cloud Computing
   - Distributed Information Systems
     - Enterprise Application Integration
   - Pervasive Systems
4. Summary

Being Open: Core Idea

- An **open distributed system** is easy to integrate with other systems
- Components may originate from:
  - Different developers
  - Different organizations
  - Different technologies
- Openness relies on the use of **standardized interfaces**

Standards: Syntax vs. Semantics

- Openness requires adherence to agreed standards
- Standards define two aspects:
  - **Syntax**:
    - How a service is accessed
    - Message formats, operation names, parameters
  - **Semantics**:
    - What a service is supposed to do
    - Meaning of operations and responses

Example: SOAP vs. REST (Illustration)

- Both **SOAP** and **REST** define open service interfaces
- **SOAP**:
  - Uses explicit interface descriptions (WSDL)
  - Relies on XML-based message envelopes
- **REST**:
  - Uses standard HTTP methods
  - Uses lightweight data formats (e.g., JSON)
- Key point: openness comes from *standards*, not from a specific technology

Interface Definition Languages (IDLs)

- Interfaces are commonly specified using **IDLs**
- IDLs precisely define **syntax**:
  - Operation names
  - Parameter and data types
  - Return values and error conditions

Interoperability and Portability

- **Interoperability**:
  - Different implementations work together
  - Rely only on shared interface definitions
- **Portability**:
  - Applications run unchanged on different systems
  - Systems implement the same interfaces

Completeness and Neutrality

- **Complete specifications**:
  - Contain all information needed to build implementations
- **Neutral specifications**:
  - Do not prescribe implementation details
  - Do not favor a specific vendor or platform
- Essential for long-term openness and evolution

From SOAP to REST: Evolution of Open Interfaces

- **Earlier approach: SOAP-based services**
  - Each service exposed explicit operations
  - Interfaces defined separately using WSDL
  - Every method had its own formal interface
  - Syntax (XML messages) and semantics were largely decoupled
- **Modern approach: REST-based services**
  - Uses a small, fixed set of standard HTTP methods
  - Same methods (`GET`, `POST`, `PUT`, `DELETE`) apply to all resources
  - Interface is uniform across services
- **Key difference**
  - REST combines **syntax and semantics** through standardized methods
  - Leads to simpler, more open, and more interoperable systems

The Core Idea: Tools vs. Rules

- Distributed systems must separate "how to do something" from "deciding when to do it."
- **Mechanism (The Tool)**:
  - The physical/technical capability.
  - Answers: *"What is possible?"*
- **Policy (The Rule)**:
  - The logic governing the capability.
  - Answers: *"How and when should we use the tool?"*

Step 1: The Mechanism (The Capability)

- Mechanisms should be general and "dumb"—they just follow orders.
- **Real Example: Smart Door Lock**
  - *Mechanism:* The hardware that can retract or extend the bolt.
  - It doesn't know *who* is at the door; it only knows how to lock/unlock.
- **Technical Example: Web Cache Storage**
  - *Mechanism:* The functions to write data to RAM or delete it.
  - It doesn't know *which* file is important; it only knows how to save/delete.

Step 2: The Policy (The Decision)

- Policies use the same mechanism to create different behaviors.
- **Smart Lock Policies**:
  - *Day Mode:* "Stay unlocked if the owner is home."
  - *Night Mode:* "Lock automatically at 10:00 PM."
- **Web Cache Policies**:
  - *LRU Policy:* "If full, delete the Least Recently Used file."
  - *FIFO Policy:* "If full, delete the oldest file first."

The Result: Modular Design

- By separating them, we get **Pluggable Systems**.
- **The "Lego" Approach**:
  - You can swap a "Night Policy" for a "Vacation Policy" without buying a new lock.
  - You can upgrade a Cache Policy to be "AI-driven" without rewriting the code that actually saves files to the disk.
- Changing a policy should be as simple as changing a configuration file.

The Trade-off: Complexity vs. Power

- **The Problem**: Total separation creates "Decision Overload."
- **The Conflict**:
  - If a system has 10 mechanisms and each has 5 policy options, the user has 50 settings to manage.
  - This is "Configuration Fatigue" (e.g., a professional server with 1,000-page manuals).
- **The Solution: Sensible Defaults**
  - Ship the system with a "Standard Policy" that works for 90% of people.
  - Keep the interface open so the 10% (experts) can plug in a custom policy.

Table of Contents

1. Introduction
2. Design Goals of Distributed Systems
   - Easy access to resources
   - Distribution Transparency
   - Being Open
     - Separating Policy from Mechanism
   - Being Scalable
3. Types of Distributed Systems
   - Distributed Computing System
     - Cluster Computing
     - Grid Computing
     - Cloud Computing
   - Distributed Information Systems
     - Enterprise Application Integration
   - Pervasive Systems
4. Summary

Why Scalability Matters

- Modern systems operate at Internet scale
- Cloud platforms serve millions of users simultaneously
- Rapid growth of mobile, IoT, and edge devices
- Systems must grow without redesign or performance collapse
- **Scalability is a fundamental design requirement**

Dimensions of Scalability

- **Size scalability** Ability to handle increasing numbers of users and resources
- **Geographical scalability** Ability to operate efficiently across large distances
- **Administrative scalability** Ability to span multiple organizations and domains

Size Scalability: Centralized Bottlenecks

- Many systems rely on centralized services
- Typically implemented as:
  - A single server
  - A tightly coupled server cluster
- Centralization leads to bottlenecks:
  - Limited CPU processing power
  - Disk and I/O contention
  - Network bandwidth saturation

Service Capacity Model

- Centralized services can be analyzed using queuing theory
- Clients submit requests to a service
- Requests wait in a queue before processing
- Increased load results in longer response times

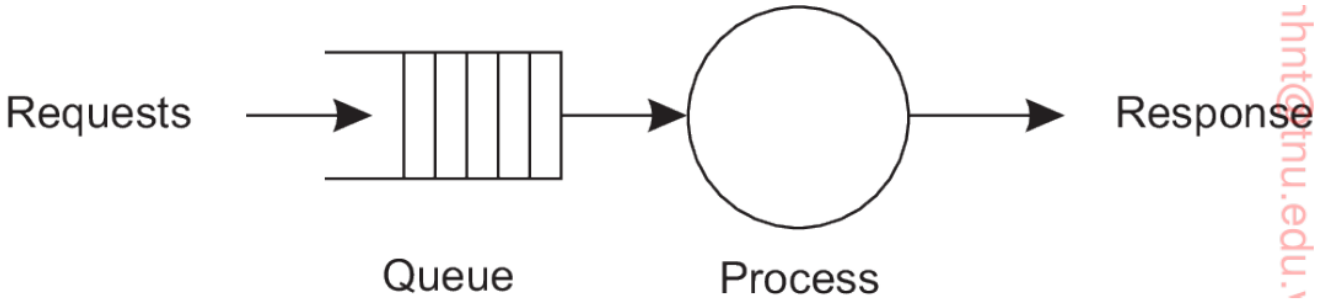


Figure 3: A centralized service modeled as a queuing system.

Geographical Scalability Challenges

- Many distributed systems rely on synchronous communication
- Performs well in local-area networks (LANs)
- Breaks down over wide-area networks (WANs) due to:
  - High communication latency
  - Limited and variable bandwidth
  - Increased probability of failures

Techniques for Hiding Latency

- **Avoid blocking remote invocations**
  - Example: Web server continues handling other requests while a database query is pending.
- **Use asynchronous communication**
  - Example: Client sends a request to a message queue and later reads the response from a reply queue.
- **Overlap computation with communication**
  - Example: Downloading file chunks while simultaneously processing earlier chunks.
- **Move computation closer to users (edge processing)**
  - Example: Edge server performing request filtering or personalization before forwarding to the cloud.

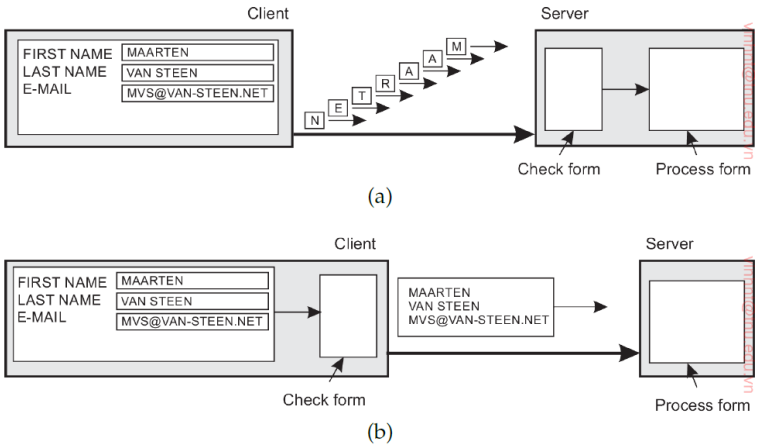
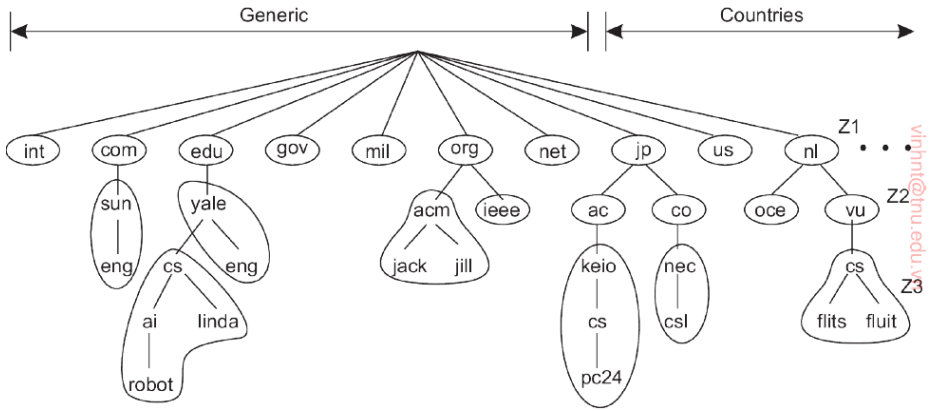


Figure 4: Latency hiding by client-side versus server-side processing.

Partitioning and Distribution

- Divide a large system into smaller components
- Distribute components across multiple machines
- Each component handles a subset of the workload
- Classic example:
  - Domain Name System (DNS)
  -
    - **Root Name Servers:** Top level of DNS hierarchy; redirect queries to appropriate TLD servers
    - **TLD Servers:** Manage domains like `.com`, `.org`, `.edu`; point to authoritative servers
    - **Authoritative Servers:** Store actual DNS records; return final IP addresses
    - **Recursive Resolvers:** Act for clients; query DNS hierarchy and cache responses



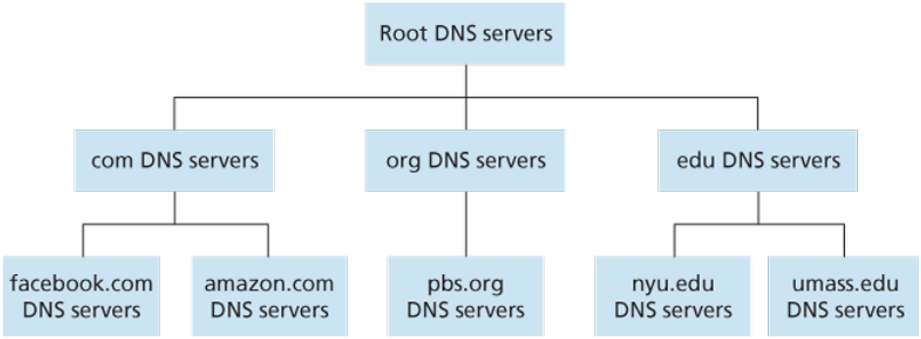


Figure 5: Partitioning the DNS namespace into hierarchical zones.

Replication, Caching, and Consistency

- **Replication** maintains multiple copies of data or services to improve availability, fault tolerance, and load balancing.
- **Caching** stores temporary local copies of data at clients or proxies to reduce access latency and decrease network and server load.
- **Consistency Challenge:** Replication and caching create multiple data copies, and maintaining strong consistency requires immediate update propagation and global agreement on operation order, which significantly limits scalability.

Administrative Scalability

- Distributed systems often span multiple organizations
- Each domain enforces its own policies
- Conflicts arise in:
  - Resource allocation
  - Security mechanisms
  - System management
- Organizations do not automatically trust users or systems from other domains

Administrative Scalability: Examples

- **Computational grids**: Resources shared across multiple institutions, each enforcing its own security and usage policies.
- **Federated clouds**: Independent cloud providers cooperate while retaining separate administrative control.
- **Scientific collaborations**: Large international research projects requiring cross-organization data and resource sharing.

Peer-to-Peer Systems and Scalability

- No centralized administrative authority, eliminating single bottlenecks
- Each node contributes resources (CPU, storage, bandwidth)
- System capacity grows naturally as more users join
- Improves scalability, fault tolerance, and load distribution
- Typical scalable applications:
  - File sharing
  - Internet telephony
  - Media streaming

Scalability: Key Takeaways

- Size scalability can be addressed with replication and partitioning
- Geographical scalability is constrained by network latency
- Administrative scalability is the most challenging
- Peer-to-peer systems demonstrate scalable, user-driven designs

Table of Contents

1. Introduction
2. Design Goals of Distributed Systems
   - Easy access to resources
   - Distribution Transparency
   - Being Open
     - Separating Policy from Mechanism
   - Being Scalable
3. Types of Distributed Systems
   - Distributed Computing System
     - Cluster Computing
     - Grid Computing
     - Cloud Computing
   - Distributed Information Systems
     - Enterprise Application Integration
   - Pervasive Systems
4. Summary

Types of Distributed Systems

- Distributed systems appear in many forms
- Broad classification:
  - **Distributed computing systems**: Focus on sharing computational power to execute large-scale or compute-intensive tasks efficiently.
  - **Distributed information systems**: Emphasize data sharing, transactions, and interoperability across multiple autonomous information sources.
  - **Pervasive systems**: Integrate computation seamlessly into the physical environment using mobile, sensor-rich, and context-aware devices.

Table of Contents

1. Introduction
2. Design Goals of Distributed Systems
   - Easy access to resources
   - Distribution Transparency
   - Being Open
     - Separating Policy from Mechanism
   - Being Scalable
3. Types of Distributed Systems
   - Distributed Computing System
     - Cluster Computing
     - Grid Computing
     - Cloud Computing
   - Distributed Information Systems
     - Enterprise Application Integration
   - Pervasive Systems
4. Summary

High-Performance Distributed Computing

- Focus on compute-intensive applications
- Two main categories:
  - Cluster computing
  - Grid computing
- Cloud computing as a logical evolution

Parallel Processing Models

- Multiprocessor systems:
  - Multiple CPUs
  - Shared physical memory
- Multicomputer systems:
  - Independent computers
  - No shared memory

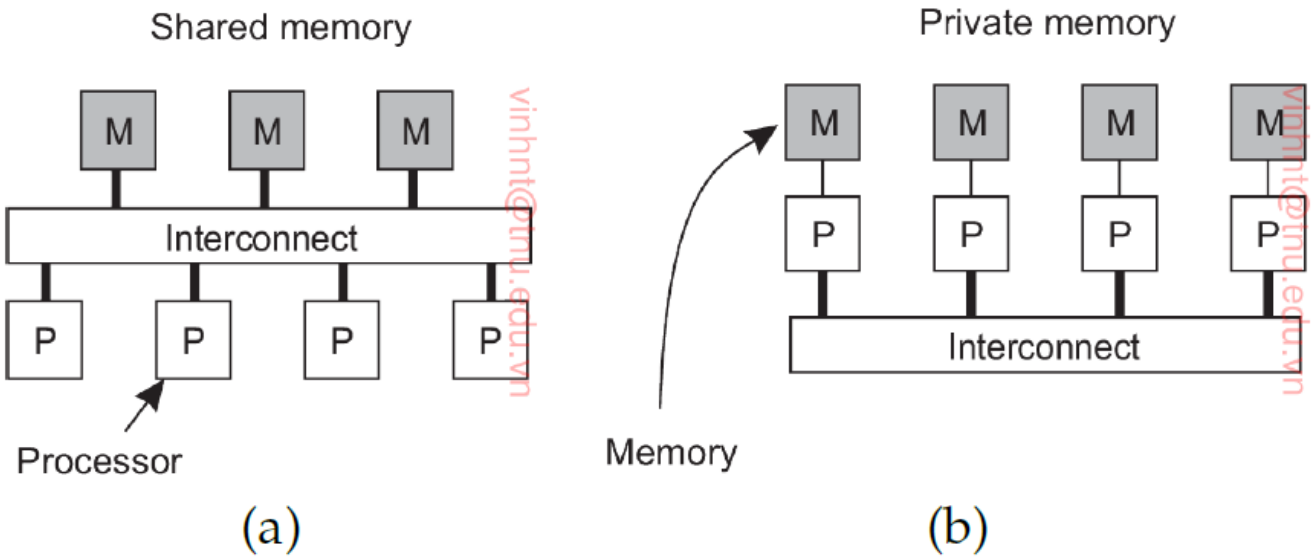


Figure 6: A comparison between (a) multiprocessor and (b) multicomputer architectures.

Distributed Shared Memory (DSM)

- Attempts to provide the illusion of shared memory on multicomputers
- Memory pages are mapped into a global virtual address space
- Page faults trigger remote page transfers over the network
- **Why DSM was abandoned:**
  - High communication latency for remote page access
  - Frequent page faults and false sharing caused excessive data movement
  - Maintaining memory consistency severely limited scalability

Cluster Computing: Overview

- Cluster computing combines multiple independent computers
- Built using:
  - Commodity PCs or workstations
  - High-speed local-area networks (LANs)
- Designed for:
  - Parallel execution of compute-intensive programs
  - High performance at relatively low cost
- Cost-effective alternative to traditional supercomputers

Beowulf Cluster Architecture

- One of the earliest and most popular cluster designs
- Typically built using Linux-based systems
- Architecture consists of:
  - **Master node**
  - **Multiple compute nodes**
- Master node responsibilities:
  - Job submission and scheduling
  - Resource allocation
  - User access and system management

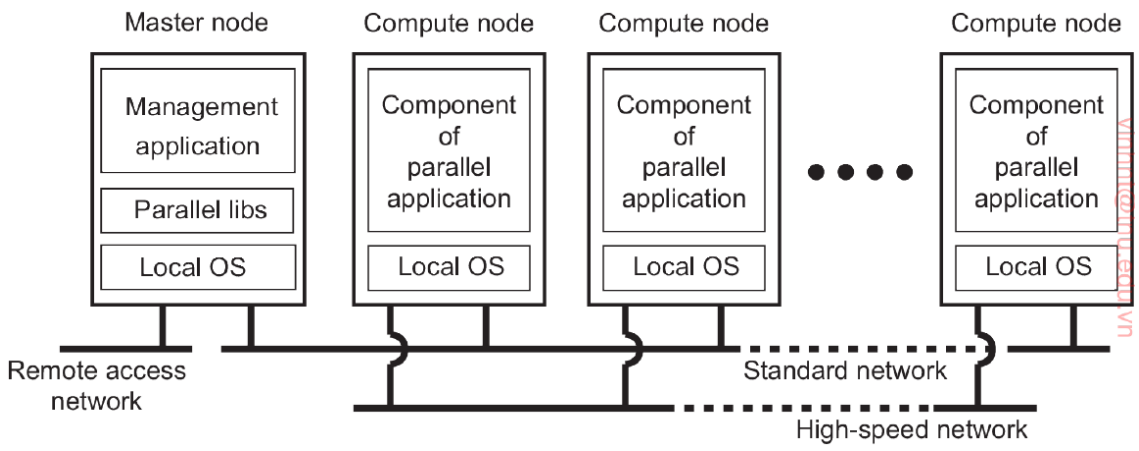


Figure 7: An example of a Beowulf-style cluster computing system.

Single-System Image (SSI)

- Goal: make the entire cluster appear as a single machine
- Users interact with the cluster as if it were one system
- Achieved through:
  - Transparent process execution: Processes run on any node without user awareness of their physical location.
  - Unified file system and resource view
- Provides a high degree of **distribution transparency**

Example: MOSIX Cluster System

- **MOSIX** is a Linux-based cluster management system
- Designed to provide a **single-system image**
- Key features:
  - Automatic and transparent process migration
  - Dynamic load balancing across cluster nodes
  - No need to modify user applications
- Processes migrate at runtime to reduce load imbalance

Why Process Migration in MOSIX?

- Different nodes may experience uneven workloads
- MOSIX monitors system load continuously
- Processes are moved to:
  - Less-loaded nodes
  - Nodes with better resource availability
- Result:
  - Improved performance
  - Better resource utilization

Modern Cluster Architectures

- Move away from fully symmetric cluster designs
- Functionality is partitioned across specialized nodes:
  - **Compute nodes**: lightweight operating systems
  - **Storage nodes**: file and directory services
  - **Management nodes**: job scheduling and monitoring
- Improves scalability, performance, and manageability

Grid Computing

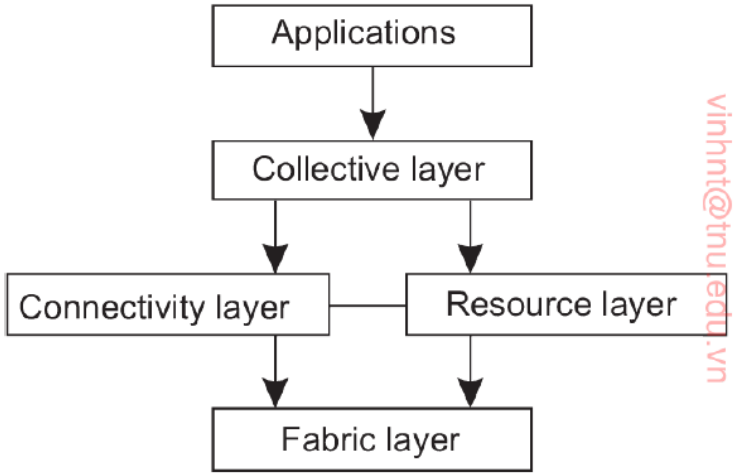
- Federation of heterogeneous systems
- No assumptions about:
  - Hardware
  - Operating systems
  - Networks
  - Administrative domains

Virtual Organizations

- Resources shared across organizations
- Users belong to virtual organizations
- Access rights defined per organization
- Resources include:
  - Compute servers
  - Storage systems
  - Databases
  - Specialized devices

Grid Computing Architecture

- Layered architecture:
  - **Fabric layer**: Provides direct access to local resources such as CPUs, storage, networks, and operating systems.
  - **Connectivity layer**: Handles communication and security, including authentication and secure data transfer.
  - **Resource layer**: Manages individual resources through allocation, monitoring, and control protocols.
  - **Collective layer**: Coordinates multiple resources for scheduling, load balancing, and directory services.
  - **Application layer**: Hosts user applications that utilize grid services without knowing resource locations.



vinhnt@tnu.edu.vn

Figure 8: A layered architecture for grid computing systems.

Cloud Computing

- Evolution of utility computing
- Virtualized resource pool
- Resources allocated dynamically
- Pay-per-use model
- Governed by Service Level Agreements (SLAs)

Cloud Architecture

- **Hardware layer**: Physical data-center resources including servers, storage devices, and networking equipment.
- **Infrastructure layer**: Virtualized resources such as virtual machines, virtual networks, and storage pools.
- **Platform layer**: Runtime environments, middleware, and development frameworks for building and deploying applications.
- **Application layer**: End-user applications delivered over the network as on-demand services.

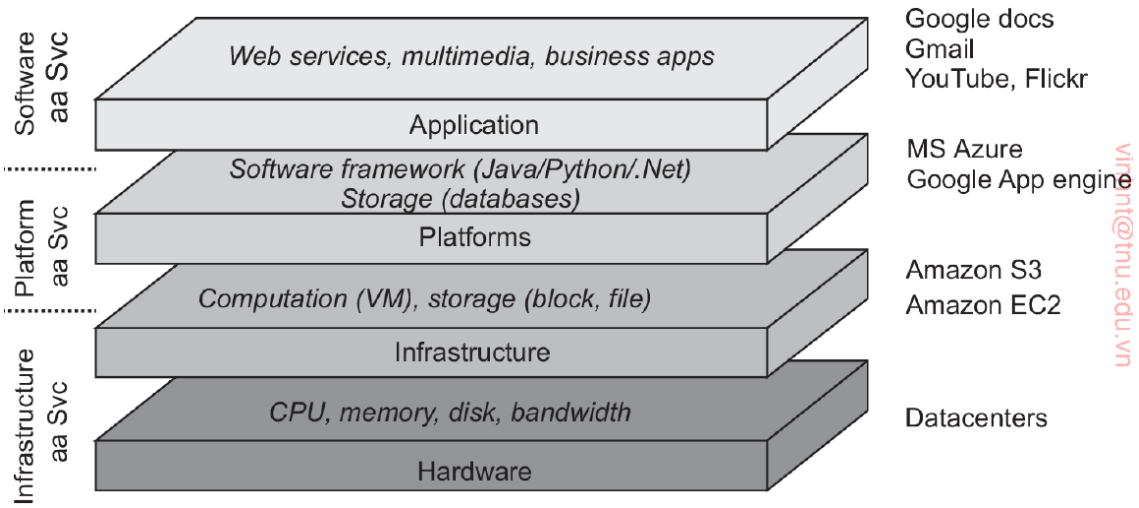


Figure 9: The organization of clouds (adapted from Zhang et al. [2010]).

Cloud Service Models

- **Infrastructure as a Service (IaaS)**: Provides virtualized computing resources such as servers, storage, and networks on demand.
- **Platform as a Service (PaaS)**: Provides development platforms and runtime environments for building and deploying applications.
- **Software as a Service (SaaS)**: Delivers complete applications to end users through a web-based interface.

Table of Contents

1. Introduction
2. Design Goals of Distributed Systems
   - Easy access to resources
   - Distribution Transparency
   - Being Open
     - Separating Policy from Mechanism
   - Being Scalable
3. Types of Distributed Systems
   - Distributed Computing System
     - Cluster Computing
     - Grid Computing
     - Cloud Computing
   - Distributed Information Systems
     - Enterprise Application Integration
   - Pervasive Systems
4. Summary

Distributed Information Systems

- Found mainly in enterprises
- Driven by need for interoperability
- Integration of heterogeneous applications
- Middleware plays a central role

Distributed Transactions

- Database operations executed as transactions
- All-or-nothing execution
- Controlled using transaction primitives

Primitive	Description
BEGIN_TRANSACTION	Mark the start of a transaction
END_TRANSACTION	Terminate the transaction and try to commit
ABORT_TRANSACTION	Kill the transaction and restore the old values
READ	Read data from a file, a table, or otherwise
WRITE	Write data to a file, a table, or otherwise

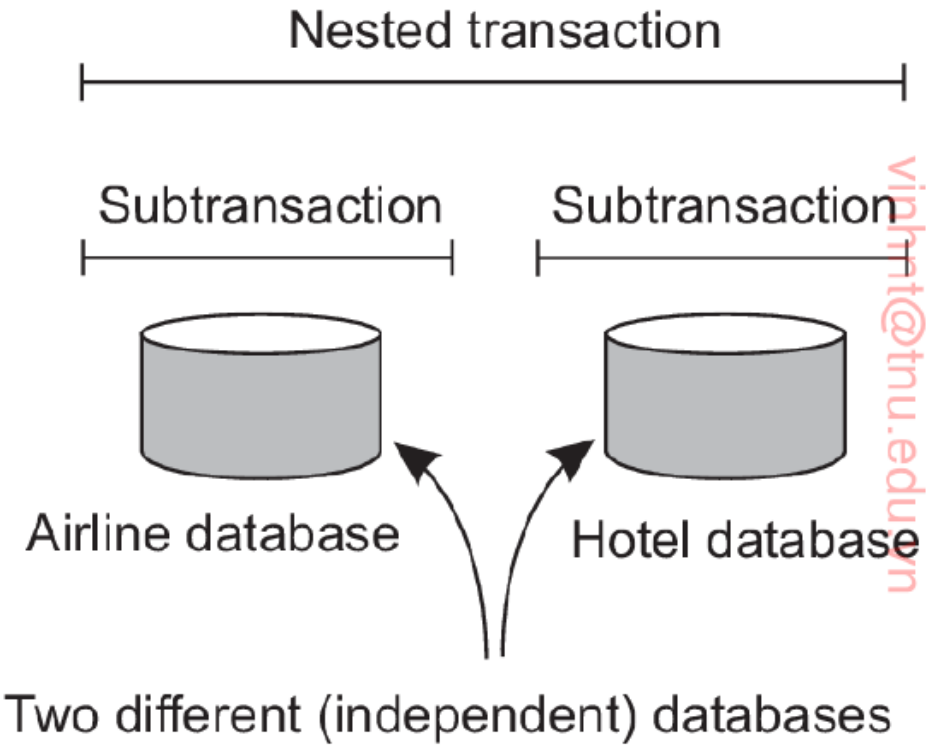
Figure 10: Example primitives for transactions

ACID Properties Transactions

- **Atomicity**: A transaction is executed entirely or not at all, with no partial effects.
- **Consistency**: A transaction transforms the system from one valid state to another, preserving all rules and constraints.
- **Isolation**: Concurrent transactions execute independently, without interfering with each other.
- **Durability**: Once a transaction commits, its effects persist despite crashes or failures.

Nested Transactions

- Transactions composed of subtransactions
- Subtransactions may run in parallel
- Only top-level commit is permanent



vinhnt@tnu.edu.vn

Figure 11: A nested transaction.

Transaction Processing Monitor

- Coordinates distributed transactions
- Implements distributed commit protocols
- Middleware service for applications

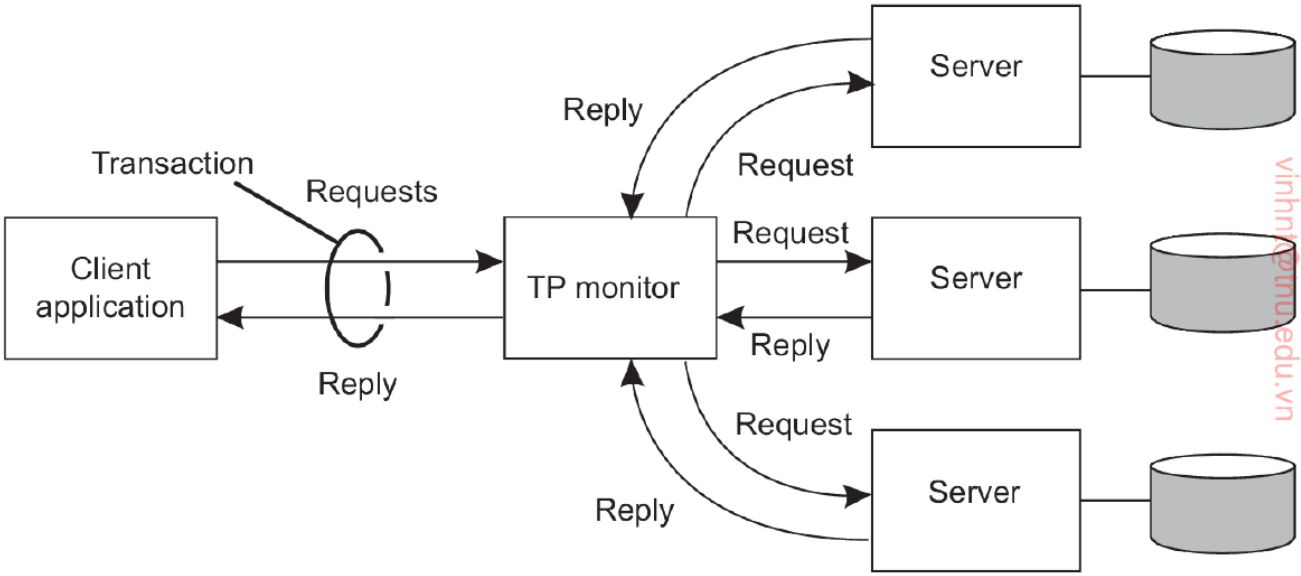
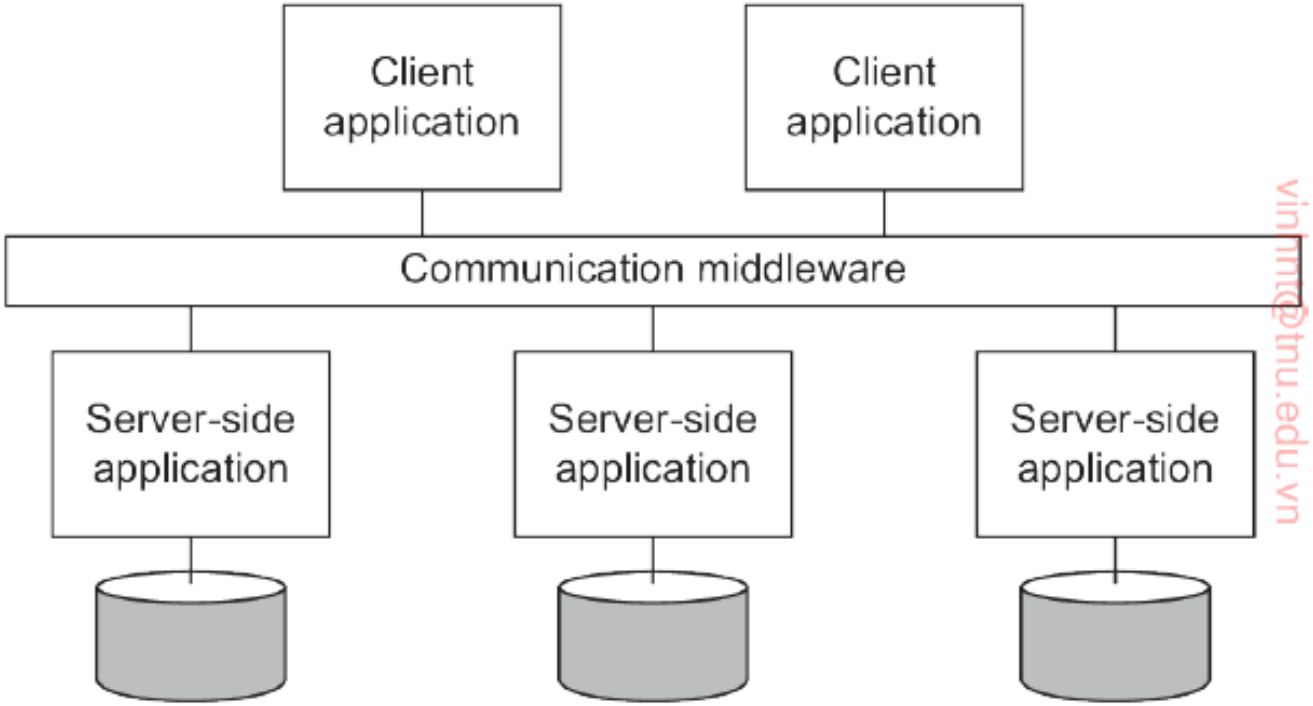


Figure 12: The role of a TP monitor in distributed systems

vinhnh@hnu.edu.vn

Enterprise Application Integration

- Integrates independent applications
- Direct communication between components
- Middleware facilitates interaction



vinhmt@tnu.edu.vn

Figure 13: Middleware as a communication facilitator in enterprise application integration.

Enterprise Application Integration: Approaches

- **File transfer**: Applications exchange data by producing and consuming shared files; simple to implement but requires agreement on file formats, storage, naming, and update notifications.
- **Shared database**: All applications access a common database using a shared schema, enabling easy data sharing and triggers, but risking schema complexity and performance bottlenecks.
- **Remote Procedure Call (RPC)**: Applications invoke operations of other applications directly, encapsulating behavior and triggering actions, but requiring both parties to be active simultaneously.
- **Messaging**: Applications communicate by exchanging messages via a middleware system, allowing asynchronous interaction and decoupling, at the cost of message handling and reliability concerns.
- Application integration is inherently complex, and middleware plays a crucial role by providing standardized RPC and messaging facilities.

Table of Contents

1. Introduction
2. Design Goals of Distributed Systems
   - Easy access to resources
   - Distribution Transparency
   - Being Open
     - Separating Policy from Mechanism
   - Being Scalable
3. Types of Distributed Systems
   - Distributed Computing System
     - Cluster Computing
     - Grid Computing
     - Cloud Computing
   - Distributed Information Systems
     - Enterprise Application Integration
   - Pervasive Systems
4. Summary

Pervasive Systems

- Computing systems seamlessly embedded into the physical environment
- Inherently distributed, adaptive, and highly dynamic
- Blurred separation between users and system components
- Interaction enabled through sensors (context capture) and actuators (feedback/action)
- Designed to be unobtrusive and context-aware

Types of Pervasive Systems

- **Ubiquitous Computing Systems**
  - Invisible computing integrated into everyday activities
- **Mobile Computing Systems**
  - Support user and device mobility with wireless connectivity
- **Sensor Networks**
  - Large collections of resource-constrained sensor nodes for monitoring and data collection

Ubiquitous Computing Requirements

- **Distribution:** Networked and geographically distributed devices cooperating transparently as a single coherent system
- **Unobtrusive Interaction:** Interaction is largely implicit and hidden, requiring minimal explicit user input
- **Context Awareness:** System adapts behavior based on contextual information such as identity, location, time, and activity
- **Autonomy:** Devices operate with minimal human intervention through self-configuration, self-management, and automatic updates
- **Intelligence:** AI-based techniques enable handling of dynamic environments, incomplete inputs, and unexpected events

Mobile Computing Systems

- Wireless communication
- Changing device locations
- Dynamic service discovery
- Disruption-tolerant networking

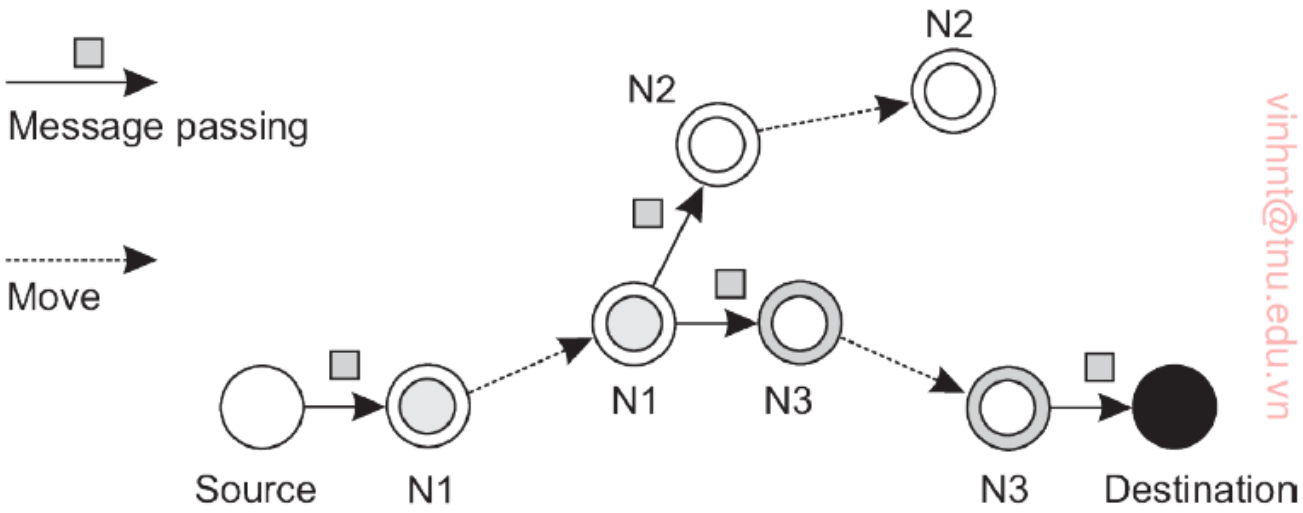


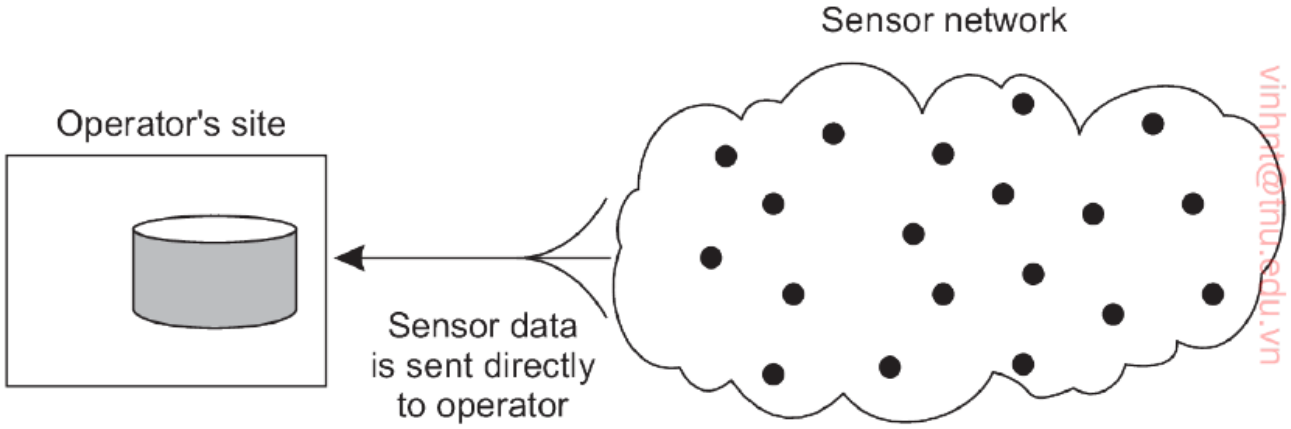
Figure 14: Passing messages in a (mobile) disruption-tolerant network.

Sensor Networks

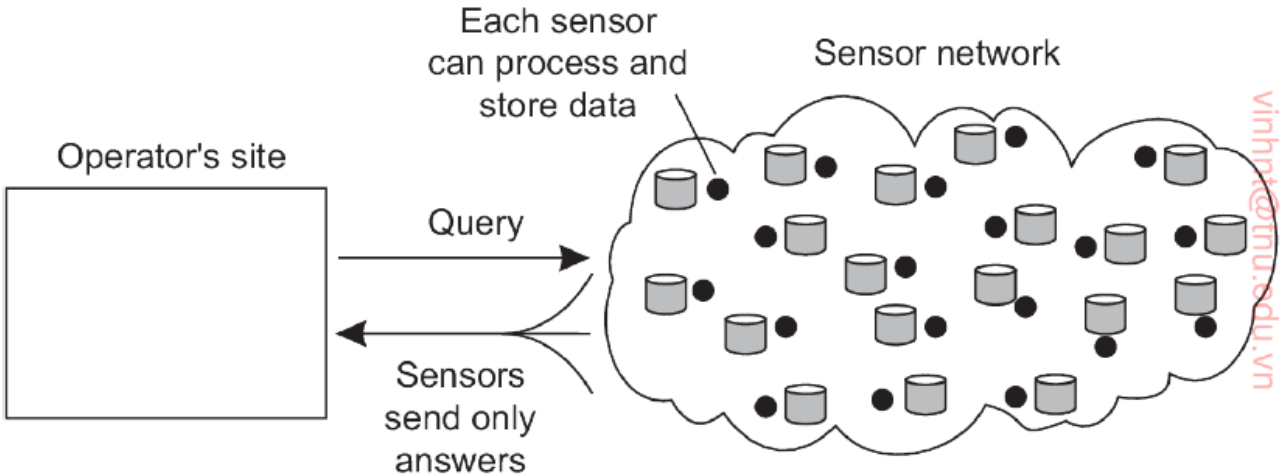
- Large number of small, battery-powered nodes
- Wireless communication
- In-network data processing
- Energy efficiency is critical

In-Network Data Processing

- Avoid centralized data collection
- Aggregate data within the network
- Tree-based query processing



(a)



vinhpt@tnu.edu.vn

Table of Contents

1. Introduction
2. Design Goals of Distributed Systems
   - Easy access to resources
   - Distribution Transparency
   - Being Open
     - Separating Policy from Mechanism
   - Being Scalable
3. Types of Distributed Systems
   - Distributed Computing System
     - Cluster Computing
     - Grid Computing
     - Cloud Computing
   - Distributed Information Systems
     - Enterprise Application Integration
   - Pervasive Systems
4. Summary

Summary: Distributed Systems Fundamentals

- Distributed systems consist of autonomous computers that cooperate
- Aim to appear as a **single coherent system**
- Middleware is the key abstraction:
  - Sits between operating systems and applications
  - Provides communication, transactions, composition, reliability
- Core design goals:
  - Resource sharing
  - Openness
  - Distribution transparency
- Full transparency is impossible and often hurts performance

Summary: Challenges and System Types

- Major challenges:
  - Trade-offs between transparency and scalability
  - Geographical and administrative scalability are hardest
  - False assumptions about networks (latency, reliability, security)
- Main classes of distributed systems:
  - Distributed computing systems (clusters, grids, clouds)
  - Distributed information systems (transactions, databases)
  - Pervasive systems (mobile systems, sensor-rich environments)